Interpretable Context Methodology (ICM)

Orchestrating AI Agent Workflows with Folders, Files, and Structure

This presentation introduces ICM — a lightweight, file-based framework that enables a single AI agent to execute complex, multi-step tasks without heavy code-based coordination frameworks.

THE PROBLEM ICM SOLVES

Most AI workflow frameworks rely on complex, code-heavy orchestration that is hard to inspect, debug, and maintain.

- Traditional AI orchestration frameworks require significant engineering overhead — custom code, APIs, and runtime state management.
- When something goes wrong, it is difficult to understand what the agent was "thinking" or which context it was operating in.
- Context pollution is a major failure mode: agents receive too much irrelevant information and produce inconsistent results.
- Scaling multi-agent systems with code-based frameworks creates tight coupling and brittle dependencies.
- There is no persistent "organizational memory" — each session starts from scratch with no record of prior decisions or workflow state.

WHAT IS INTERPRETABLE CONTEXT METHODOLOGY?

ICM is a framework that uses folder structures, markdown files, and local scripts to orchestrate AI agent workflows — making every decision and context boundary visible and readable.

INTERPRETABILITY

The word "interpretable" is central: every routing decision, context boundary, and workflow stage is expressed as a readable file.

- No custom runtime
- No complex API calls
- Agent navigates by reading files

CONTEXT ISOLATION

The folder structure itself *is* the workflow engine: folder names, file names, and file contents collectively define what the agent should do and where.

- Each workspace loads only needed context
- Prevents prompt pollution

PERSISTENT MEMORY

ICM supports persistent organizational memory. Decisions, standards, state, and workflow history are stored as files that survive across sessions.

CORE DESIGN PRINCIPLES

ICM is governed by five architectural principles that ensure deterministic, maintainable, and scalable AI execution.

Local Context Authority	The nearest context file is authoritative — local rules always override global defaults.
Explicit Ownership	Each workspace owns its own workflows, outputs, state, standards, and deliverables.
Deterministic Routing	The agent should never guess which workspace to use — the CONTEXT.md file makes routing unambiguous.
Minimal Prompt Pollution	Agents load only the context necessary for the current task — no irrelevant data.
Interpretability	The repository structure itself explains ownership, workflow, routing, and execution flow to both humans and AI.

These principles are implemented through a consistent set of files that appear in every ICM workspace.

THE ICM FILE ANATOMY

Every ICM workspace is defined by three core markdown files that together tell an AI agent exactly what to do, where to work, and when to escalate.

ICM.md

THE FRAMEWORK CONTRACT

- Defines the workspace purpose and philosophy
- Specifies which files the agent must load before acting
- Lists what the agent must avoid (unrelated workspaces, archived projects, irrelevant data)
- Defines escalation conditions (conflicting requirements, missing approvals, ambiguous behavior)

CONTEXT.md

THE ROUTING & BOUNDARY MAP

- Declares the workspace mission and ownership scope
- Lists allowed actions (create files, generate deliverables, update workflows)
- Lists forbidden actions (modify unrelated workspaces, fabricate outputs, bypass review)

AGENT.md

THE OPERATIONAL CONTRACT

- Defines inputs the agent accepts (requirements, specifications, tasks)
- Defines outputs the agent produces (deliverables, artifacts, workflow updates)

THE TWO ICM TEMPLATES

The ICM template library includes two base templates, each designed for a different class of project, plus an advanced overlay for stage-gated pipelines. Deliverables come from the companion SE-Deliverables library.

generic-agent-oriented-ICM

A generalized multi-agent project structure suitable for content workflows, software projects, business operations, documentation systems, and general-purpose orchestration.

DEFAULT WORKSPACES

writing-room/
Tutorials, documentation, content

production/
Implementation, software builds, engineering

community/
Outreach, marketing, customer communication

systems-engineering-ICM

A domain-specific structure for engineering-intensive projects: automation systems, DAQ systems, hardware/software integration, validation-heavy projects, and engineering governance.

DEFAULT WORKSPACES

source-development/
Software, automation, instrumentation

documentation/
Manuals, plans, validation docs

requirements/
Capture, baseline, trace

sales/
Proposals, technical collateral, presentations

SUPPORT DIRECTORIES

standards/, templates/, state/, decisions/, risk-management/, compliance/, configuration-management/, metrics/, governance/, verification-validation/

AGENT EXECUTION FLOW

An AI agent operating inside an ICM project follows a strict, deterministic sequence of file reads before taking any action.

01 LOAD CORE CONTEXT

Read the top-level `ICM.md` to understand the project framework and philosophy.

02 READ GLOBAL ROUTING

Read the top-level `CONTEXT.md` to determine which workspace handles the current task.

03 ROUTE TO WORKSPACE

Navigate to the correct workspace folder based on the routing rules.

04 LOAD LOCAL CONTEXT

Read the workspace-level `CONTEXT.md`, `ICM.md`, and `AGENT.md`.

05 LOAD SUPPORTING CONTEXT

Read applicable standards, workflow metadata, and active project state files.

06 EXECUTE WITHIN SCOPE

Perform only actions permitted by the local context; never modify unrelated workspaces.

07 ESCALATE WHEN NEEDED

If requirements conflict, approvals are missing, or behavior is ambiguous, escalate rather than guess.

KEY INSIGHT: This sequence ensures that every agent action is traceable to a specific file that authorized it — making the system fully auditable.

CREATING A PROJECT INSTANCE

Creating an ICM project instance is a structured, collaborative process between a human and an AI agent — the human selects the template and provides project details; the AI generates the customized structure.

01 SELECT A TEMPLATE

Choose **generic-agent-oriented-ICM** for general projects or **systems-engineering-ICM** for engineering-intensive work.

02 PROVIDE TEMPLATE TO AI

Point the AI agent at AI_PROJECT_CREATION_INSTRUCTIONS.md.

03 AI INTERVIEWS THE USER

The AI asks one question at a time, presenting options, to gather: project name, purpose, domain, technical stack, workflow stages, organizational structure, documentation requirements, and compliance constraints.

04 AI GENERATES INSTANCE

The AI produces: customized folder structure, populated ICM.md / CONTEXT.md / AGENT.md files, standards, templates, workflow scaffolds, state tracking files, README files, and placeholder project artifacts.

05 PACKAGE & VERSION CONTROL

A local AI agent creates the project directly as a sibling folder under .ai/ (ZIP archive only in chat environments). The human initializes a git repository for version control to maintain organizational memory.

RECOMMENDED FOLDER STRUCTURE

A .ai/ parent directory scopes AI access to a single directory tree and provides a consistent home for your ICM templates, shared resources, and all generated project instances.

```
.ai/
├── ICM/
│   └── Templates/
│       ├── README.md
│       ├── generic-agent-oriented-ICM/
│       ├── systems-engineering-ICM/
│       └── advanced-options/
├── SE-Deliverables/
├── skills/
├── memory/
├── tools/
├── my-first-project/
└── my-second-project/
```

.ai/

Scope AI access to this directory tree. The name signals where AI-assisted work lives to both humans and tools.

ICM/

This templates library. Clone it once and reuse it across all your projects. SE-Deliverables/ is its optional companion — DoD DID digests, manual templates, and UAT artifacts.

skills/ & memory/ & tools/

Shared resources not tied to any single project: shared prompts, persistent cross-project context, and reusable scripts.

Project folders

Generated ICM instances live here as their own git repositories, sibling to ICM/ — not inside it.

EXPECTED OUTPUT ARTIFACTS

A fully generated ICM project instance is a complete, self-describing operational system — not just a folder structure, but a living workflow engine.

`ICM.md (per workspace)`

Framework contract and context loading rules.

`CONTEXT.md (per workspace)`

Routing map, ownership boundaries, allowed/forbidden actions.

`AGENT.md (per workspace)`

Inputs, outputs, and definition of done.

`README.md (per directory)`

Human-readable explanation of each area.

`standards/`

Coding standards, naming conventions, compliance rules.

`templates/`

Reusable document and artifact templates.

`state/`

Persistent operational memory — active project state.

`decisions/`

Decision log for traceability and audit.

`workflows/`

Numbered, ordered workflow stage folders (e.g., 01-requirements/).

`workspace/`

Active working area for in-progress artifacts.

KEY INSIGHT: The resulting repository is both human-readable and AI-operable — it serves simultaneously as documentation, a workflow engine, and organizational memory.

ICM AS ORGANIZATIONAL MEMORY

ICM repositories are not just project scaffolds — they are long-term operational assets that accumulate knowledge, decisions, and workflow history across the life of a project.

state/ provides **persistent operational memory** — the AI agent can resume work across sessions without losing context.

decisions/ creates a **traceable decision log** — every significant choice is recorded with its rationale, supporting audit and governance requirements.

standards/ encodes **organizational knowledge** — coding standards, naming conventions, and compliance rules are stored as files the agent can read and enforce.

Version control transforms the ICM repository into a **full audit trail** — every change to context, workflow, or standards is tracked over time.

ICM repositories should be treated as **engineering assets** — they are operational systems, not disposable scaffolds.

RECOMMENDED PRACTICE

```
git init
# review .gitignore first — a file
# committed once stays in history
git status
git add .
git commit -m "Initial ICM instance"
```

ICM QUALITY RULES

A correctly generated ICM project instance must satisfy seven quality criteria that ensure it remains deterministic, maintainable, and scalable over time.

DETERMINISTIC

The same task always routes to the same workspace — no ambiguity.

MAINTAINABLE

Files are structured so humans and AI can update them without breaking routing.

SCALABLE

New workspaces and workflows can be added without restructuring existing ones.

MINIMAL AMBIGUITY

Every folder name, file name, and routing rule has a single clear meaning.

FUTURE AUTOMATION READY

The structure supports scheduled tasks, multi-agent handoffs, and CI/CD integration.

HUMAN READABLE

Any team member can open the repository and understand the project structure.

MULTI-AGENT CAPABLE

Multiple AI agents can operate in different workspaces simultaneously without conflict.

WHAT TO AVOID

Contradictory workflows, ambiguous ownership, hidden routing rules, and undocumented standards are the four failure modes that make an ICM project unmaintainable.

GETTING STARTED WITH ICM

ICM is immediately usable with the provided templates — getting started requires only a template selection, a brief AI interview, and a git repository.

IMMEDIATE ACTIONS

1. **Choose your template** — generic-agent-oriented-ICM for general projects; systems-engineering-ICM for engineering work.
2. **Open an AI agent session** — point the AI agent at AI_PROJECT_CREATION_INSTRUCTIONS.md.
3. **Answer the AI's questions** — project name, purpose, domain, technical stack, workflow stages.
4. **Review the generated instance** — verify routing is unambiguous and ownership is explicit.
5. **Initialize version control** — run git init, review the generated .gitignore against your stack, check git status, then make the first commit.
6. **Operate the project** — point AI agents at the repository; they will self-route using context files.

LOOKING AHEAD

- Advanced options add stage-gated pipelines; formal deliverables come from the companion SE-Deliverables library.
- The same framework scales from single-agent projects to complex multi-agent systems.
- ICM repositories become more valuable over time as they accumulate decisions, standards, and workflow history.

CORE PROMISE OF ICM

A single AI agent can execute complex, multi-step tasks simply by reading files — no custom code, no heavy frameworks, no opaque runtime state.